

# Промежуточный отчёт по лабораторной работе №2 по предмету “Технологии управления данными”

## Текст задачи:

Лабораторная работа №2. Разработка модели и скрипта создания хранилища данных.

Хранилище данных должно реализовано с использованием реляционной СУБД, если иное не было оговорено заранее. Архитектура хранилища должна соответствовать одной из устоявшихся практик: Кимбалл, Инмон, Якорная модель, Data vault, или иная практика по согласованию с преподавателем. Студент обязан в отчете обосновать выбранную архитектуру. Наполнение будет производится в автоматизированном режиме из данных, поступающих из источников, разработанных в лабораторной работе №1. Для хранилища данных предусмотрена только операция добавления данных, операции модификации и удаления недопустимы.

Пример: Сформировать модель «звезда», в которой основной таблицей фактов будет продажа конкретного товар, измерениями будут товары, покупатели и т.д.

На основании имеющейся информации можно создать реляционную модель, утвердить ее у преподавателя. На основании утвержденной модели создать даталогическую модель, утвердить ее у преподавателя.

На основании созданных моделей создать скрипт создания хранилища данных, проверить его работоспособность. Создать тестовый (!) скрипт наполнения хранилища, проверить работоспособность.

Решение:

Выбранная реляционная СУБД - PostgreSQL.

Выбранная архитектура - модель “Звезда”, из-за простоты и скорости разработки, простоты презентации данных (SQL-запросов) и простоты добавления новых фактов и измерений.

Диаграмма связей сущностей:

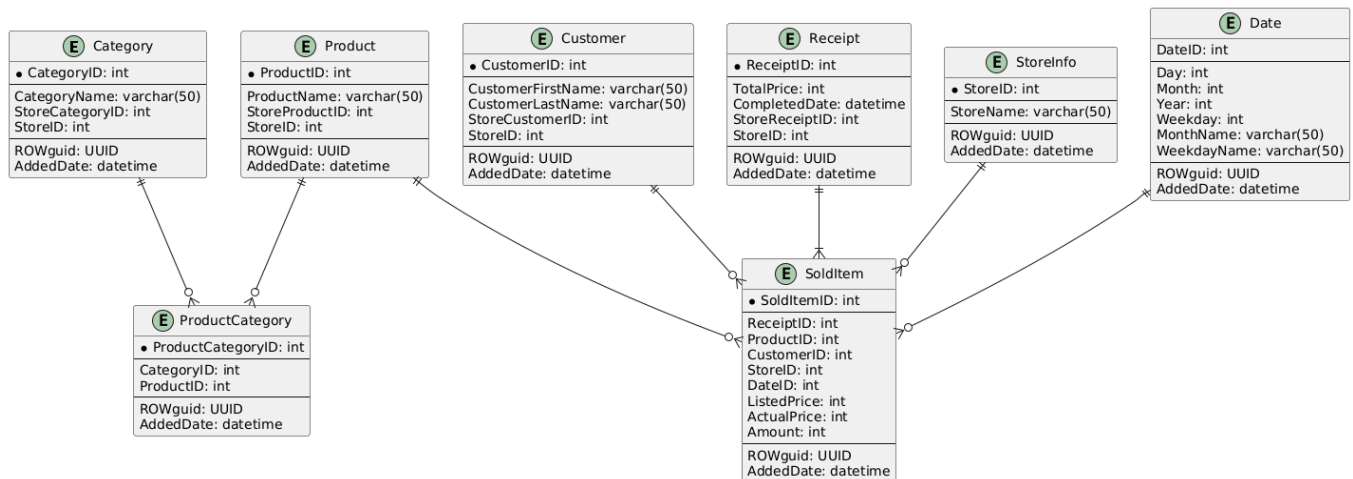


Схема “Звезда”, где таблица фактов - продажа товара, измерения - категория-продукт, покупатель, чек, информация о филиале, дата.

Store<Entity>ID - изначальный первичный ключ сущности в базе данных филиала (обозначаемый через StoreID)

SoldItem - продаже товара конкретному покупателю в конкретном магазине по конкретному чеку.

Date - сущность “Дата”.

Category - сущность “Категория”, содержит также название категории.

Product - сущность “Товар”, содержит название товара и цену товара.

ProductCategory - сущность “Категория товара”, обеспечивает связь многие-ко-многим между Товаром и Категорией

Customer - сущность “Покупатель”, содержит имя и фамилию покупателя.

Receipt - сущность “Сделка” (чек), содержит полную цену чека, дату совершения сделки.

StoreInfo - информация о магазине (филиале).

Скрипты и код:

 entities\_creation.sql  
 seeding.py

entities\_creation.sql - скрипт создания базы данных (в соответствии по диаграмме и описанию выше):

```
CREATE TABLE IF NOT EXISTS StoreInfo (  
    ROWguid UUID DEFAULT gen_random_uuid(),  
    AddedDate TIMESTAMP DEFAULT now(),  
  
    StoreID INTEGER PRIMARY KEY,  
    StoreName VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS Category (  
    ROWguid UUID DEFAULT gen_random_uuid(),  
    AddedDate TIMESTAMP DEFAULT now(),  
  
    CategoryID INTEGER PRIMARY KEY,  
    CategoryName VARCHAR(50) NOT NULL,  
    StoreCategoryID INTEGER NOT NULL,  
    StoreID INTEGER NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS Customer (  
    ROWguid UUID DEFAULT gen_random_uuid(),  
    AddedDate TIMESTAMP DEFAULT now(),  
  
    CustomerID INTEGER PRIMARY KEY,  
    StoreCustomerID INTEGER NOT NULL,  
    StoreID INTEGER NOT NULL,  
    CustomerFirstName VARCHAR(50) NOT NULL,  
    CustomerLastName VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS Product (  
    ROWguid UUID DEFAULT gen_random_uuid(),  
    AddedDate TIMESTAMP DEFAULT now(),  
  
    ProductID INTEGER PRIMARY KEY,  
    ProductName VARCHAR(50) NOT NULL,  
    StoreProductID INTEGER NOT NULL,  
    StoreID INTEGER NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS ProductCategory (  
    ROWguid UUID DEFAULT gen_random_uuid(),  
    AddedDate TIMESTAMP DEFAULT now(),  
  
    ProductCategoryID INTEGER PRIMARY KEY,  
    CategoryID INTEGER NOT NULL,  
    ProductID INTEGER NOT NULL,  
  
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID),
```

```

    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);

CREATE TABLE IF NOT EXISTS Receipt (
    ROWguid UUID DEFAULT gen_random_uuid(),
    AddedDate TIMESTAMP DEFAULT now(),

    ReceiptID INTEGER PRIMARY KEY,
    StoreReceiptID INTEGER NOT NULL,
    StoreID INTEGER NOT NULL,
    TotalPrice INTEGER NOT NULL,
    CompletedDate TIMESTAMP NOT NULL
);

CREATE TABLE IF NOT EXISTS Date (
    ROWguid UUID DEFAULT gen_random_uuid(),
    AddedDate TIMESTAMP DEFAULT now(),

    DateID INTEGER PRIMARY KEY,
    Day INTEGER NOT NULL,
    Month INTEGER NOT NULL,
    Year INTEGER NOT NULL,
    Weekday INTEGER NOT NULL,
    MonthName VARCHAR(50) NOT NULL,
    WeekdayName VARCHAR(50) NOT NULL
);

CREATE TABLE IF NOT EXISTS SoldItem (
    ROWguid UUID DEFAULT gen_random_uuid(),
    AddedDate TIMESTAMP DEFAULT now(),

    SoldItemID INTEGER PRIMARY KEY,
    ReceiptID INTEGER NOT NULL,
    CustomerID INTEGER NOT NULL,
    ProductID INTEGER NOT NULL,
    StoreID INTEGER NOT NULL,
    DateID INTEGER NOT NULL,
    ListedPrice INTEGER NOT NULL,
    ActualPrice INTEGER NOT NULL,
    Amount INTEGER NOT NULL,

    FOREIGN KEY (ReceiptID) REFERENCES Receipt(ReceiptID),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (StoreID) REFERENCES StoreInfo(StoreID),
    FOREIGN KEY (DateID) REFERENCES Date(DateID)
);

```

seeding.py - скрипт полного переноса данных из баз данных из лабораторной №1 (без урегулирования конфликтов), написанный на языке Python:

```
import psycopg2

db_config_west = {
    "dbname": "west_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5446"
}

db_config_east = {
    "dbname": "east_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5447"
}

db_config_warehouse = {
    "dbname": "warehouse_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5448"
}

conn_west = psycopg2.connect(**db_config_west)
cur_west = conn_west.cursor()
conn_east = psycopg2.connect(**db_config_east)
cur_east = conn_east.cursor()
conn_warehouse = psycopg2.connect(**db_config_warehouse)
cur_warehouse = conn_warehouse.cursor()

curs = [cur_east, cur_west]
stores = [(0, "east"), (1, "west")]
for ID, name in stores:
    cur_warehouse.execute("INSERT INTO StoreInfo (StoreID, StoreName) VALUES (%s, %s)",
                           (str(ID), name))

productID = 0
categoryID = 0
productcategoryID = 0
product_map = dict()
category_map = dict()
for storeID in range(2):
```

```

cur = curs[storeID]
cur.execute("SELECT ProductID, ProductName, ProductPrice FROM Product")
products = cur.fetchall()
for product in products:
    ID = str(product[0])
    name = str(product[1])
    cur_warehouse.execute("INSERT INTO Product (ProductID, ProductName,
StoreProductID, StoreID) VALUES (%s, %s, %s, %s)",
                           (productID, name, ID, storeID))
    product_map[(int(ID), storeID)] = productID
    productID += 1
cur.execute("SELECT CategoryID, CategoryName FROM Category")
categories = cur.fetchall()
for category in categories:
    ID = str(category[0])
    name = str(category[1])
    cur_warehouse.execute("INSERT INTO Category (CategoryID,
CategoryName, StoreCategoryID, StoreID) VALUES (%s, %s, %s, %s)",
                           (categoryID, name, ID, storeID))
    category_map[(int(ID), storeID)] = categoryID
    categoryID += 1
cur.execute("SELECT CategoryID, ProductID FROM ProductCategory")
product_categories = cur.fetchall()
for product_category in product_categories:
    cID = category_map[(product_category[0], storeID)]
    pID = product_map[(product_category[1], storeID)]
    cur_warehouse.execute(
        "INSERT INTO ProductCategory (ProductCategoryID, CategoryID,
ProductID) VALUES (%s, %s, %s)",
        (productcategoryID, cID, pID))
    productcategoryID += 1

customerID = 0
receiptID = 0
itemID = 0
dates = set()
months = ["january", "february", "march", "april", "may", "june", "july",
"august", "september", "october", "november", "december"]
weekdays = ["monday", "tuesday", "wednesday", "thursday", "friday",
"saturday", "sunday"]
for storeID in range(2):
    cur = curs[storeID]
    cur.execute("SELECT CustomerID, CustomerFirstName, CustomerLastName FROM
Customer")
    customers = cur.fetchall()
    for customer in customers:
        cID = customer[0]
        first_name = customer[1]

```

```

        last_name = customer[2]
        cur_warehouse.execute("INSERT INTO Customer (CustomerID,
StoreCustomerID, StoreID, CustomerFirstName, CustomerLastName) VALUES (%s,
%s, %s, %s, %s)",
                                (customerID, cID, storeID, first_name,
last_name))
        cur.execute("SELECT ReceiptID, TotalPrice, CompletedDate FROM
Receipt WHERE CustomerID = %s",
                    (cID,))
        receipts = cur.fetchall()
        for receipt in receipts:
            rID = receipt[0]
            total_price = receipt[1]
            completed_date = receipt[2]
            cur_warehouse.execute("INSERT INTO Receipt (ReceiptID,
StoreReceiptID, StoreID, TotalPrice, CompletedDate) VALUES (%s, %s, %s, %s,
%s)",
                                (receiptID, rID, storeID, total_price,
completed_date))
            cur.execute("SELECT ProductID, ActualPrice, Amount FROM
ReceiptItem WHERE ReceiptID = %s",
                        (rID,))
            items = cur.fetchall()
            for item in items:
                product = item[0]
                pID = product_map[(product, storeID)]
                price = item[1]
                amount = item[2]
                cur.execute("SELECT CompletedDate FROM Receipt WHERE
ReceiptID = %s",
                            (rID,))
                completed_date = cur.fetchall()[0][0]
                year = completed_date.year
                month = completed_date.month
                day = completed_date.day
                weekday = completed_date.weekday()
                weekdayname = weekdays[weekday]
                monthname = months[month - 1]
                date = f'{year:04}-{month:02}-{day:02}'
                dateID = f'{year:04}{month:02}{day:02}'
                if date not in dates:
                    dates.add(date)
                    cur_warehouse.execute(
                        "INSERT INTO Date (DateID, Day, Month, Year,
Weekday, MonthName, WeekdayName)"
                        " VALUES (%s, %s, %s, %s, %s, %s, %s)",
                        (dateID, day, month, year, weekday, monthname,
weekdayname))

```

```

        cur.execute("SELECT ProductPrice FROM Product WHERE
ProductID = %s",
                    (product,))
        listed_price = cur.fetchall()[0]
        if cur is cur_east:
            store = 0
        elif cur is cur_west:
            store = 1
        else:
            raise IndexError("invalid store")
        cur_warehouse.execute("INSERT INTO SoldItem (SoldItemID,
ReceiptID, CustomerID, ProductID, StoreID, DateID, ListedPrice,
ActualPrice, Amount)"
                               " VALUES (%s, %s, %s, %s, %s, %s, %s,
%s, %s)",
                               (itemID, receiptID, customerID, pID,
store, dateID, listed_price, price, amount))
        itemID += 1
        receiptID += 1
        customerID += 1

conn_west.commit()
conn_east.commit()
conn_warehouse.commit()
cur_west.close()
cur_east.close()
cur_warehouse.close()
conn_west.close()
conn_east.close()
conn_warehouse.close()
print("All done!")

```